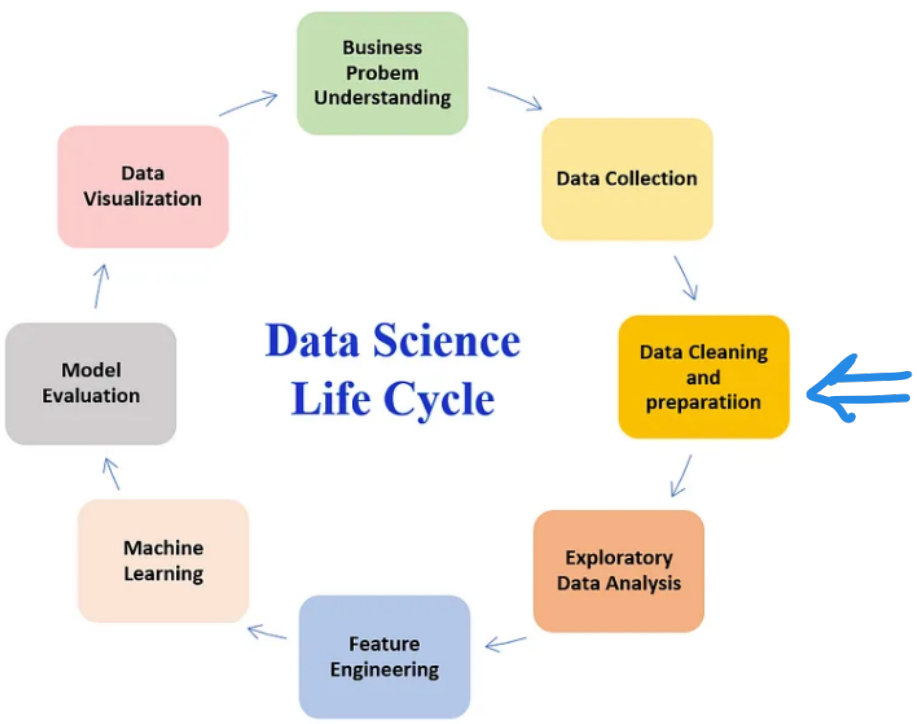
CSC 251 Lecture 16 – Data Cleaning and Date Formats

Data Science Life Cycle



https://mustafaserdarkonca.medium.com/data-science-life-cycle-e4d74afe4bf5

Cleaning: fixing or removing incorrect, corrupted, incorrectly formatted, duplicate, or incomplete data

Munging: changing data into another format or arrangement so that it can be used or processed

Wrangling – munging, data viz, aggregation, training a statistical model

"Tidy" Data

According to Hadley Wickham, an expert in the R language, tidy data can be described as meeting the following criteria:

- Each row is an observation
- Each column is a variable
- Each type of observational unit forms a table

Most of the data you've seen was been "tidy", rectangular already cleaned!

What is Missing Data

Missing data can be displayed as: `NAN, NaN, nan`

Compare with Python's `None`

What do these represent? ~~true~~ ~~false~~ ~~0~~ ~~""~~ null !

(demo in **Lecture16Code.ipynb**)

Where do they come from?

1) Empty cells

	A	B	C	D
1	petname	months	weight	rabies
2	Clifford	4	45.2	yes
3	Clifford	9	67.4	yes
4	Clifford	12		yes
5	Clifford	14	101.3	
6	Clifford	24	121.3	yes

```
pets2 = pd.read_csv('morepets.csv')
```

	petname	months	weight	rabies
0	Clifford	4	45.2	yes
1	Clifford	9	67.4	yes
2	Clifford	12	NaN	yes
3	Clifford	14	101.3	NaN
4	Clifford	24	121.3	yes

data never entered

2) Merged data

3) Re-indexing

Recall the gapminder data set:

```
life_exp = df.groupby(['year'])['lifeExp'].mean()
```

```
year
1952    49.057620
1957    51.507401
1962    53.609249
1967    55.678290
1972    57.647386
1977    59.570157
1982    61.533197
1987    63.212613
1992    64.160338
1997    65.014676
2002    65.694923
2007    67.007423
Name: lifeExp, dtype: float64
```

```
life = life_exp.reindex(range(2000,2010))
```

```
year
2000          NaN
2001          NaN
2002    65.694923
2003          NaN
2004          NaN
2005          NaN
2006          NaN
2007    67.007423
2008          NaN
2009          NaN
Name: lifeExp, dtype: float64
```

4) Propagation through calculations

What if values are missing in the Gapminder data?

continent	count	mean	std	min	25%	50%	75%	max
Africa	624.0	48.865330	9.150210	23.599	42.37250	47.7920	54.41150	76.442
Americas	300.0	64.658737	9.345088	37.579	58.41000	67.0480	71.69950	80.653
Asia	396.0	60.064903	11.864532	28.801	51.42625	61.7915	69.50525	82.603
Europe	360.0	71.903686	5.433178	43.585	69.57000	72.2410	75.45050	81.757
Oceania	24.0	74.326208	3.795611	69.120	71.20500	73.6650	77.55250	81.235

To count the missing values:

```
np.count_nonzero(life.isnull())
8
```

Luckily, NumPy and pandas ignores missing values when determining descriptive statistics!

Could be an issue with counts or when plotting!

Data Cleaning - Missing Data

Some options include:

1) Recode/replace the nan's

```
pets2.fillna(0)
```

	petname	months	weight	rabies
0	Clifford	4	45.2	yes
1	Clifford	9	67.4	yes
2	Clifford	12	NaN	yes
3	Clifford	14	101.3	NaN
4	Clifford	24	121.3	yes

no change?

returns a new df

No change in the original df? To correct this:

```
pets2.fillna(0, inplace = True)
```

or change old one

or

```
newpets = pets2.fillna(0)
```

	petname	months	weight	rabies
0	Clifford	4	45.2	yes
1	Clifford	9	67.4	yes
2	Clifford	12	0.0	yes
3	Clifford	14	101.3	0
4	Clifford	24	121.3	yes

Is this a good idea?

Maybe... maybe not!

2) Fill forward/backwards/interpolate:

`pets2.ffill()` or `pets2.bfill()` or `pets2.interpolate(method='linear')`

	petname	months	weight	rabies
0	Clifford	4	45.20	yes
1	Clifford	9	67.40	yes
2	Clifford	12	84.35	yes
3	Clifford	14	101.30	NaN
4	Clifford	24	121.30	yes

assumes a linear growth curve

3) Drop the missing values:

```
pets_dropna = pets2.dropna()
```

	petname	months	weight	rabies
0	Clifford	4	45.2	yes
1	Clifford	9	67.4	yes
4	Clifford	24	121.3	yes

reduces the amount of data...

Advantages? Disadvantages?

Data Cleaning – Duplicates

What if our DataFrame includes duplicate observations? Is this a problem?

	petname	months	weight
0	Spot	4	2.1
1	Spot	8	5.8
2	Spot	10	7.5
3	Spot	14	8.9
4	Spot	18	9.2
5	Spot	10	7.5
6	Lassie	4	6.0
7	Lassie	8	15.1
8	Lassie	12	21.9
9	Lassie	14	23.0
10	Lassie	16	24.1

. . .

obvious duplicates - not 2 different office visits

We can save the dataframe with duplicates removed. What else changes?

```
nodup = dfd.drop_duplicates()
nodup
```

	petname	months	weight
0	Spot	4	2.1
1	Spot	8	5.8
2	Spot	10	7.5
3	Spot	14	8.9
4	Spot	18	9.2
6	Lassie	4	6.0
7	Lassie	8	15.1
8	Lassie	12	21.9
9	Lassie	14	23.0
10	Lassie	16	24.1

...

To reset the index: (`inplace = True` will move the old index to a column)

```
nodup.reset_index(inplace=True)
```

And then rename the column:

```
new = nodup.rename(columns={"index": "patient_id"})
```

`nodup`

	index	petname	months	weight
0	0	Spot	4	2.1
1	1	Spot	8	5.8
2	2	Spot	10	7.5
3	3	Spot	14	8.9
4	4	Spot	18	9.2
5	6	Lassie	4	6.0
6	7	Lassie	8	15.1
7	8	Lassie	12	21.9
8	9	Lassie	14	23.0
9	10	Lassie	16	24.1
10	11	Fido	3	10.2

`new`

	patient_id	petname	months	weight
0	0	Spot	4	2.1
1	1	Spot	8	5.8
2	2	Spot	10	7.5
3	3	Spot	14	8.9
4	4	Spot	18	9.2
5	6	Lassie	4	6.0
6	7	Lassie	8	15.1
7	8	Lassie	12	21.9
8	9	Lassie	14	23.0
9	10	Lassie	16	24.1
10	11	Fido	3	10.2

Data Arrangement Issues

1) Columns should contain variables, not values, "long" data versus "wide" data
Suppose you combine the information from the following "wide" tables:

```
name    jan    feb    mar    apr  ... dec
jim     $100   $200   $200   $300      $300
mike    $200   $250   $50    $100      $170
```

wide - generally more human-readable

```
name    jan    feb    mar    apr  ... dec
jim     10     10     10     30        30
mike    5      20     7      25        5
```

long - better for analysis

To create this "long" table. Notice the repetitive values in the first column!

```
name    month    kpi      value

jim     jan      amount   100
jim     feb      amount   120
...
jim     jan      items    10
jim     feb      items    10
...
mike    dec      amount   170
...
mike    dec      items    5
```

Some options:

A) Reshape the DataFrame using pivot or transpose

`df.T` or `df.transpose()`

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
index	0	1	2	3	4	6	7	8	9	10	11	12	13	14	15
petname	Spot	Spot	Spot	Spot	Spot	Lassie	Lassie	Lassie	Lassie	Lassie	Fido	Fido	Fido	Fido	Fido
months	4	8	10	14	18	4	8	12	14	16	3	8	10	14	18
weight	2.1	5.8	7.5	8.9	9.2	6.0	15.1	21.9	23.0	24.1	10.2	64.1	89.3	99.3	102.1

B) Use aggregation methods with `groupby`

Aggregation	Description
`count()`	Total number of items
`first()`, `last()`	First and last item
`mean()`, `median()`	Mean and median
`min()`, `max()`	Minimum and maximum
`std()`, `var()`	Standard deviation and variance
`mad()`	Mean absolute deviation
`prod()`	Product of all items
`sum()`	Sum of all items

	Country.Name	year	VarA	VarB
1	United Kingdom	2017	2.69902e+12	35.1
2	United States	2017	1.94796e+13	41.2
3	Mexico	2017	1.15891e+12	NA
4	United Kingdom	2018	2.90079e+12	NA
5	United States	2018	2.05272e+13	41.4
6	Mexico	2018	1.22241e+12	46.7
7	United Kingdom	2019	2.87867e+12	NA
8	United States	2019	2.13726e+13	41.5
9	Mexico	2019	1.26940e+12	NA

	Country.Name	VarA2017	VarA2018	VarA2019	VarA2020	VarA2021	VarB2017	VarB2018	VarB2019	VarB2020	VarB2021
1	United Kingdom	2.69902e+12	2.90079e+12	2.87867e+12	2.75690e+12	3.18686e+12	35.1	NA	NA	NA	NA
2	United States	1.94796e+13	2.05272e+13	2.13726e+13	2.08937e+13	2.29961e+13	41.2	41.4	41.5	NA	NA
3	Mexico	1.15891e+12	1.22241e+12	1.26940e+12	1.08712e+12	1.29304e+12	NA	46.7	NA	45.4	NA

2) Columns contain multiple variables

	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	Jae Crowder	Boston Celtics	99.0	SF	25.0	6-6	235.0	Marquette	6796117.0
3	R.J. Hunter	Boston Celtics	28.0	SG	22.0	6-5	185.0	Georgia State	1148640.0
6	Jordan Mickey	Boston Celtics	55.0	PF	21.0	6-8	235.0	LSU	1170960.0
7	Kelly Olynyk	Boston Celtics	41.0	C	25.0	7-0	238.0	Gonzaga	2165160.0

. . .

Split first and last names into two columns of a new dataframe:

```
new = data["Name"].str.split(" ", n=1, expand = True)
```

	0	1
0	Avery	Bradley
1	Jae	Crowder
3	R.J.	Hunter
6	Jordan	Mickey
7	Kelly	Olynyk

. . .

Create new columns in the original DataFrame, drop the combined name column:

```
data["First Name"] = new[0]
data["Last Name"] = new[1]
data.drop(columns=["Name"], inplace=True)
```

	Team	Number	Position	Age	Height	Weight	College	Salary	First Name	Last Name
0	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0	Avery	Bradley
1	Boston Celtics	99.0	SF	25.0	6-6	235.0	Marquette	6796117.0	Jae	Crowder
3	Boston Celtics	28.0	SG	22.0	6-5	185.0	Georgia State	1148640.0	R.J.	Hunter
6	Boston Celtics	55.0	PF	21.0	6-8	235.0	LSU	1170960.0	Jordan	Mickey
7	Boston Celtics	41.0	C	25.0	7-0	238.0	Gonzaga	2165160.0	Kelly	Olynyk

Data Type Clashes

Check type, be aware of categorical types which are often used for space efficiency.

```
df.dtypes
last_name            object
first_name         category
birthday     datetime64[ns]
gender             category
type               category
state              category
party              category
dtype: object
```

Solution: If categorical, convert to string object:

```
df['colname_str'] = df['colname'].astype(str)
```

Date Formats

Google says: “There is no standard format for representing dates worldwide, so dates should be stored externally as part of textual data.”

Some common date formats include:

- **YYYYMMDD**: Four-digit year, two-digit month, two-digit day
- **DDMMYYYY**: Two-digit day, two-digit month, four-digit year
- **MMDDYYYY**: Two-digit month, two-digit day, four-digit year
- **DDMMYY**: Two-digit day, two-digit month, last two digits of year
- **MDY**: Month, day, year
- **DMY**: Day, month, year
- **YMD**: Year, month, day
- **JUL**: yy/ddd
- **ISO**: YYYY-MM-DD
- **USA**: MM/DD/YYYY
- **EUR**: DD. MM. YYYY
- **JIS**: YYYY-MM-DD

Many languages use a **datetime** data type to store an instant in time.

Python datetime module:

```
datetime.date, datetime.time, date.today, today.year, today.month. . .
```

`pandas.to_datetime` converts an argument to a datetime object:

```
date = "10/30/2024"
pd.to_datetime(date)                    Timestamp('2024-10-30 00:00:00')
```

To extract the date:

```
str(date2)[:10]                         '2024-10-30'
```

To convert a dataframe column:

```
df['date'] = pd.to_datetime(df['date'])
```